



Phase 4 — Project Planning Phase

Development Timeline, Sprint Schedule & Milestone Targets

Planning documentation for an AI-powered debt relief and
financial recovery platform

Prepared as part of the project documentation set
Document type: Delivery Timeline & Sprint Planning Overview

Phase 4: Project Planning Phase

This document details the development timeline, sprint schedule, and milestone targets for the platform — covering the order in which the backend, AI service, frontend, and testing work were planned and delivered across six two-week sprints.

1. Project Milestones

Seven milestones mark the delivery of each major functional layer, from database foundation through to a tested, deployed product.

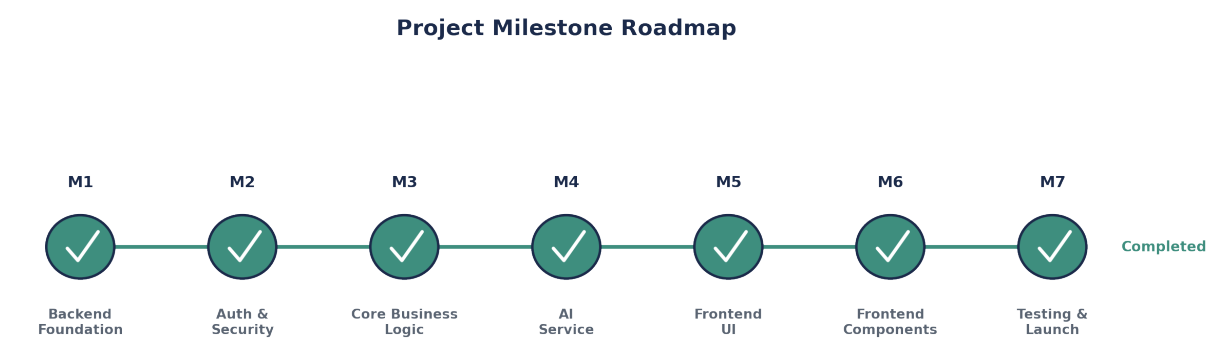


Figure 1. Milestone roadmap — all seven milestones completed.

Milestone	Target Deliverable	Status
Milestone 1	Backend Foundation: SQLite connection, SQLAlchemy tables, and Pydantic schemas.	Completed
Milestone 2	Auth & Security: Direct bcrypt password hashing and JWT token exchange.	Completed
Milestone 3	Core Business Logic: Ratios and rule-based settlement prediction calculators.	Completed
Milestone 4	AI Service: Google Gemini integration and local fallback mockup engine.	Completed
Milestone 5	Frontend UI: React Vite initialization and Vanilla CSS styling system.	Completed
Milestone 6	Frontend Components: Dashboard dials, predictor logs, and letter generators.	Completed

Milestone	Target Deliverable	Status
Milestone 7	Testing & Launch: Python pytest unit testing and production deployment.	Completed

2. Sprint Schedule Overview

Work was organized into six sequential, two-week sprints, moving from backend foundations to frontend polish and finally to testing and deployment.

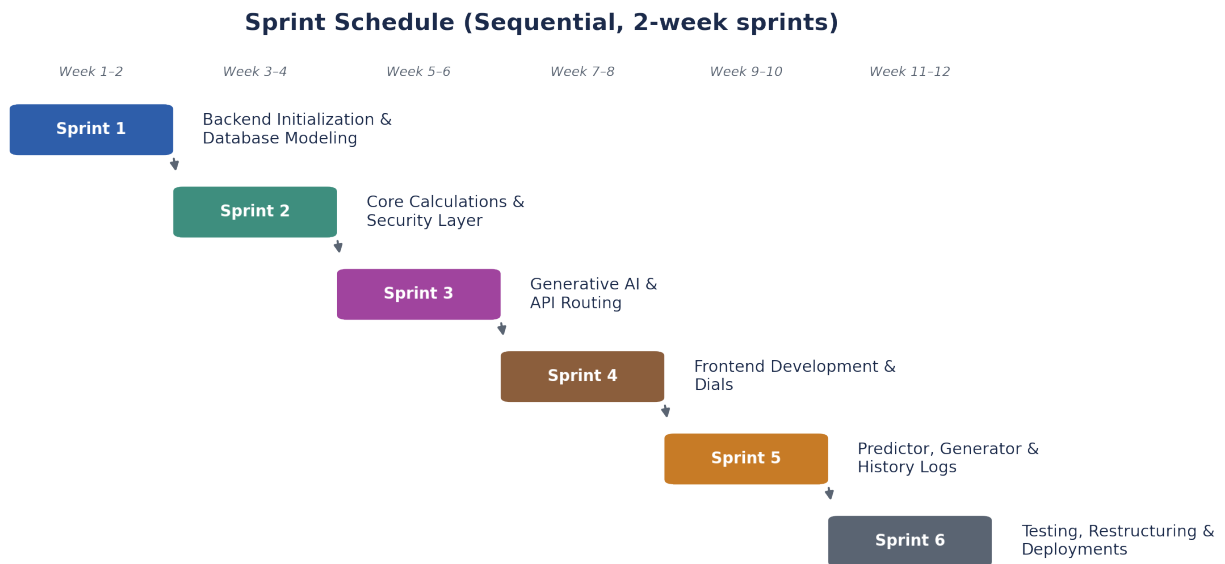


Figure 2. Sequential sprint schedule across the six-sprint delivery cycle.

3. Work Breakdown Structure (WBS)

Each sprint's scope is broken down below into its concrete engineering tasks.

Sprint 1 Backend Initialization & Database Modeling

- Set up FastAPI folder structure, define Python dependency packages, and initialize a virtual environment (venv).
- Establish configuration scripts (config.py) to read environment variables and API keys.
- Define SQLAlchemy database models (models.py) reflecting the six target relational tables.
- Establish session makers and database engines inside database.py.

Sprint 2 Core Calculations & Security Layer

- Write mathematical algorithms inside financial_engine.py to evaluate monthly surpluses, DTI ratios, and EMI parameters.
- Integrate direct bcrypt password verification and generate token structures inside auth.py.
- Implement JWT-header parsing dependencies to authenticate API endpoints.

Sprint 3 Generative AI & API Routing

- Integrate the Google Gemini SDK inside gemini_service.py to compile letters and strategies in JSON format.
- Write the rule-based local backup text generator to handle offline runs without crashing.
- Declare CRUD API routers inside main.py covering user profiles, loan items, predictions, and negotiation history logs.

Sprint 4 Frontend Development & Dials

- Scaffold the React project using Vite and write global token CSS scripts inside index.css.
- Build the AuthContext to handle register actions and persist login keys.
- Develop user pages: Login, Register, and Dashboard.
- Write custom SVG elements on the dashboard to render the animated debt stress level gauge.

Sprint 5 Predictor, Generator & History Logs

- Develop the SettlementPredictor page to show target savings and payment timeline structures.
- Develop the LetterGenerator page with dropdowns for hardships and buttons to copy or download output files.
- Develop the History page utilizing expanding accordion containers for historical audits.

Sprint 6 Testing, Restructuring & Deployments

- Write unit tests in pytest to verify math boundaries (Low vs Severe stress indices).
- Compile React assets using npm run build to verify syntax and path configurations.
- Deploy backend to Railway, configure environment variables, and create public DNS endpoints.

4. Additional Notes & Recommendations

The following points are not part of the original planning text, but are common considerations worth tracking as the plan moves from schedule into execution and, eventually, maintenance.

4.1 Risk Register

A short list of the risks most likely to affect this timeline, and how each is mitigated.

Risk	Likelihood	Impact	Mitigation
Gemini API rate limits or outages during letter generation	Medium	High	Local rule-based fallback generator (already scoped in Sprint 3).
SQLite concurrency limits under multi-user load	Medium	Medium	Plan early migration path to PostgreSQL via SQLAlchemy.
JWT/session handling bugs exposing user data across accounts	Low	High	Dedicated auth test suite; row-level checks on every query.
Frontend/backend schema drift (React expects fields API no longer sends)	Medium	Medium	Shared Pydantic/OpenAPI contract reviewed each sprint.
Deployment environment variables misconfigured on launch	Low	Medium	Deployment checklist and staging dry-run before go-live.

4.2 Definition of Done (per Sprint)

- Code merged to the main branch with no failing pytest or build checks.
- New endpoints documented in the auto-generated Swagger UI.
- Manual smoke test performed against the feature's primary user flow.
- Environment variables and secrets updated in every environment that needs them (local, staging, production).

4.3 Suggested Roles per Sprint Focus

- Sprints 1–2 (backend, security): benefit most from a backend/Python-leaning contributor.
- Sprint 3 (Gen-AI, routing): benefits from whoever owns prompt design alongside backend integration.
- Sprints 4–5 (frontend, dials, generator UI): benefit from a frontend/React-leaning contributor.
- Sprint 6 (testing, deployment): a good fit for whoever is most comfortable owning CI/CD and infra.

4.4 Suggested Next Steps

- Phase 5: Post-launch monitoring plan — uptime checks, error logging, and Gemini API usage dashboards.
- Phase 6: User feedback loop — lightweight in-app survey or feedback form after letter generation.
- Phase 7: Roadmap for v2 features (e.g. multi-lender bulk negotiation, email delivery of letters).
- Ongoing: Sprint retrospectives after each cycle to capture velocity and refine future sprint sizing.